



RAPPORT

Cryptographie & sécurité des réseaux LAN

Réalisé Par :

▪ Mouhamadou Falilou Sarr

Sous la direction :

▪ Mr Fall

Juin 2024

INTRODUCTION

La cryptographie et la sécurité des réseaux locaux (LAN) jouent un rôle crucial dans la protection des données et des communications au sein des organisations modernes. Ce rapport explore ces deux domaines essentiels, en mettant l'accent sur les principes théoriques et les applications pratiques qui garantissent la confidentialité, l'intégrité et l'authenticité des informations. Nous commencerons par une étude détaillée des fonctions de hachage et de l'intégrité des données, suivie d'une analyse comparative de la cryptographie symétrique et asymétrique, avec des exemples concrets d'algorithmes et de cas d'utilisation. Ensuite, nous aborderons la configuration des VPN, tant pour l'accès à distance que pour les connexions site-à-site, en utilisant des équipements Cisco et des serveurs Ubuntu. Enfin, nous décrirons la mise en place d'une infrastructure à clé publique (PKI) avec OpenSSL, incluant la création et la gestion de certificats numériques. Ce rapport vise à fournir une compréhension approfondie et pratique des mécanismes de sécurité essentiels pour protéger les réseaux LAN contre les menaces potentielles.

PARTIE 1 : Cryptographie

I- HASHING ET INTÉGRITÉ DES DONNÉES



Qu'est-ce qu'une fonction de hachage et pourquoi est-elle utilisée en cryptographie ?

Une fonction de hachage prend une entrée de longueur variable et produit une sortie de longueur fixe, appelée "haché". Elle est utilisée en cryptographie pour assurer l'intégrité des données, car une petite modification de l'entrée produit un haché complètement différent, ce qui permet de détecter les altérations.



Exemple de fonction de hachage : SHA-256

SHA-256 est une fonction de hachage couramment utilisée. Elle prend des données de longueur variable et produit un haché de 256 bits. Elle est utilisée pour vérifier l'intégrité des données, par exemple, pour comparer le haché d'un fichier téléchargé avec le haché fourni pour vérifier que le fichier n'a pas été altéré.

II- CRYPTOGRAPHIE SYMÉTRIQUE ET ASYMÉTRIQUE



Différences entre la cryptographie symétrique et asymétrique :

Cryptographie symétrique : Utilise une seule clé pour le chiffrement et le déchiffrement.

Exemple : AES (Advanced Encryption Standard).

Cas d'utilisation typique : chiffrement de données en masse.

Cryptographie asymétrique : Utilise une paire de clés (publique et privée).

Exemple : RSA (Rivest-Shamir-Adleman).

Cas d'utilisation typique : échanges sécurisés de clés, signatures numériques.



Générons une paire de clés RSA de 4096 bits :

- Générons la clé privée :

```
root@ubuntu:/home/falilou/Bureau# openssl genpkey -algorithm RSA -out private_key.pem -pkeyopt rsa_keygen_bits:4096
.....+++++
.....+++++
root@ubuntu:/home/falilou/Bureau#
```

- Générons la clé publique :

```
root@ubuntu:/home/falilou/Bureau# openssl rsa -pubout -in private_key.pem -out public_key.pem
writing RSA key
root@ubuntu:/home/falilou/Bureau#
```

- Chiffrons un fichier avec la clé publique :

```
root@ubuntu:/home/falilou/Bureau# openssl rsautl -encrypt -inkey public_key.pem -pubin -in crypto.txt -out encrypted.txt
root@ubuntu:/home/falilou/Bureau#
```

- Déchiffrons le fichier chiffré avec la clé privée :

```
root@ubuntu:/home/falilou/Bureau# openssl rsautl -decrypt -inkey private_key.pem -in encrypted.txt -out decrypted.txt
root@ubuntu:/home/falilou/Bureau#
```

- Signons un document avec la clé privée :

```
root@ubuntu:/home/falilou/Bureau# openssl dgst -sha256 -sign private_key.pem -out signature.bin crypto.txt
root@ubuntu:/home/falilou/Bureau#
```

- Vérifions la signature avec la publique :

```
root@ubuntu:/home/falilou/Bureau# openssl dgst -sha256 -verify public_key.pem -signature signature.bin crypto.txt
Verified OK
root@ubuntu:/home/falilou/Bureau#
```



Générons une clé secrète aléatoire :

```
root@ubuntu:/home/falilou/Bureau# openssl rand -hex 32 > secret.key
root@ubuntu:/home/falilou/Bureau#
```

- Chiffrons un message avec AES-256-CBC :

```
root@ubuntu:/home/falilou/Bureau# openssl enc -aes-256-cbc -salt -in crypto.txt -out encrypted_message.bin -pass file:./secret.key
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.
root@ubuntu:/home/falilou/Bureau#
```

- Déchiffrons le message avec AES-256-CBC :

```
root@ubuntu:/home/falilou/Bureau# openssl enc -d -aes-256-cbc -in encrypted_message.bin -out decrypted_message.txt -pass file:./secret.key
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.
root@ubuntu:/home/falilou/Bureau#
```

PARTIE 2 : SÉCURITÉ DES LAN

I- VPN D'ACCÈS AVEC CISCO



Description d'une configuration d'un VPN d'accès (Remote Access VPN) sur un routeur Cisco :

- Configuration l'interface externe :

```
interface GigabitEthernet0/1
ip address 203.0.113.1 255.255.255.0
no shutdown
```

- **Activation du serveur VPN :**

crypto isakmp policy 1

encr aes

authentication pre-share

group 2

- **Définir une clé pré-partagée :**

crypto isakmp key vpnkey address 0.0.0.0 0.0.0.0

- **Configuration des paramètres IPsec :**

crypto ipsec transform-set VPN-SET esp-aes esp-sha-hmac

crypto dynamic-map VPN-MAP 10

set transform-set VPN-SET

- **Application de la configuration IPsec à l'interface :**

crypto map VPN-MAP

interface GigabitEthernet0/1

crypto map VPN-MAP

Configuration de l'adresse du pool VPN :

ip local pool VPNPOOL 192.168.1.1 192.168.1.10

- **Configuration de l'accès du VPN :**

crypto isakmp client configuration group VPNGROUP

key vpnkey

pool VPNPOOL

- **Installer le client VPN :**

```
sudo apt-get install vpnc
```

- **Créer un fichier de configuration VPN et ajouter le contenu suivant :**

```
IPSEC gateway 203.0.113.1
```

```
IPSEC ID VPNGROUP
```

```
IPSEC secret vpnkey
```

```
XAUTH username myusername
```

```
XAUTH password mypassword
```

- **Démarrer la connexion VPN**

```
sudo vpnc myvpn.conf
```

II- VPN SITE-TO-SITE AVEC CISCO :

Expliquons comment configurer un VPN Site-to-Site entre deux routeurs Cisco :

1. **Politiques de sécurité IPSec :** On configure les politiques de sécurité IPSec sur chaque routeur pour spécifier les algorithmes de chiffrement, d'authentification, et les paramètres de sécurité utilisés pour établir la connexion VPN.
2. **Clé pré-partagée :** On définit une clé pré-partagée sur chaque routeur. Cette clé est utilisée pour authentifier les communications entre les deux extrémités du tunnel VPN.
3. **Configuration du tunnel IPSec :** On configure le tunnel IPSec sur chaque routeur en définissant les transform-sets (ensemble de transformations) qui spécifient les algorithmes de chiffrement et

d'authentification utilisés pour sécuriser le trafic VPN. Configurez également les ACL (listes d'accès) pour spécifier quel trafic doit être crypté et envoyé à travers le tunnel.

4. **Association à l'interface** : On associe la configuration du VPN (carte crypto) à l'interface réseau par laquelle le trafic VPN sera envoyé (par exemple, FastEthernet0/0).
5. **Vérification et débogage** : On utilise des commandes de vérification telles que `show crypto isakmp sa` et `show crypto ipsec sa` pour vérifier l'état du tunnel IPsec et s'assurer que le trafic est correctement chiffré et transmis.



Fournissons une configuration complète pour les deux côtés du tunnel VPN :



- Configurons les adresses IP sur l'interface du routeur 1 :

```
R1#conf t
Enter configuration commands, one per line. End with CNTL/Z.
R1(config)#int FastEthernet0/0
R1(config-if)# ip address 192.168.1.1 255.255.255.0
R1(config-if)# duplex auto
R1(config-if)# speed auto
R1(config-if)#
```

- Configurons les adresses IP sur l'interface du routeur 2 :

```
R2#conf t
Enter configuration commands, one per line. End with CNTL/Z.
R2(config)#int FastEthernet0/0
R2(config-if)# ip address 192.168.2.1 255.255.255.0
R2(config-if)# duplex auto
R2(config-if)# speed auto
R2(config-if)#
R2(config-if)#
```

- Configuration du routeur 1 :
- ❖ Configuration des politiques de sécurité ISAKMP/IKE :

```
R1(config)#crypto isakmp policy 10
R1(config-isakmp)# encryption aes
R1(config-isakmp)#hash sha
R1(config-isakmp)#authentication pre-share
R1(config-isakmp)# group 2
R1(config-isakmp)# lifetime 86400
R1(config-isakmp)#
```

encryption aes : Utilisation de l'algorithme de chiffrement AES.

hash sha : Utilisation de l'algorithme de hachage SHA-1.

authentication pre-share : Authentification par clé pré-partagée.

group 2 : Utilisation du groupe de sécurité DH (Diffie-Hellman) 2.

lifetime 86400 : Durée de vie de la phase IKE en secondes (86400 secondes = 24 heures).

- ❖ Configuration de la clé pré-partagée :

```
R1(config)#crypto isakmp key passer address 192.168.2.1
R1(config)#
```

- ❖ Configuration du tunnel IPSec :

```
R1(config)#crypto ipsec transform-set TRANSFORM_SET esp-aes esp-sha-hmac
R1(cfg-crypto-trans)#crypto map VPN_MAP 10 ipsec-isakmp
% NOTE: This new crypto map will remain disabled until a peer
and a valid access list have been configured.
R1(config-crypto-map)#set peer 192.168.2.1
R1(config-crypto-map)#set transform-set TRANSFORM_SET
R1(config-crypto-map)#match address VPN_ACL
R1(config-crypto-map)#
```

- ❖ Configuration de l'ACL pour le tunnel VPN :

```
R1(config)#$ 100 permit ip 192.168.1.0 0.0.0.255 192.168.2.0 0.0.0.255
R1(config)#
```

- ❖ Association à l'interface sortante :

```
R1(config)#int FastEthernet0/0
R1(config-if)#crypto map VPN_MAP
R1(config-if)#
*Mar  1 00:28:19.139: %CRYPTO-6-ISA_KMP_ON_OFF: ISAKMP is ON
R1(config-if)#
```


- ❖ Configuration des politiques de sécurité ISAKMP/IKE :

```
R2(config)#crypto isakmp policy 10
R2(config-isakmp)#encryption aes
R2(config-isakmp)#hash sha
R2(config-isakmp)#authentication pre-share
R2(config-isakmp)#group 2
R2(config-isakmp)#lifetime 86400
R2(config-isakmp)#
```

- ❖ Configuration de la clé pré-partagée :

```
R2(config)#crypto isakmp key passer address 192.168.1.1
R2(config)#
```

- ❖ Configuration du tunnel IPsec :

```
R2(config)#crypto ipsec transform-set TRANSFORM_SET esp-aes esp-sha-hmac
R2(cfg-crypto-trans)#crypto map VPN_MAP 10 ipsec-isakmp
% NOTE: This new crypto map will remain disabled until a peer
      and a valid access list have been configured.
R2(config-crypto-map)#set peer 192.168.1.1
R2(config-crypto-map)#set transform-set TRANSFORM_SET
R2(config-crypto-map)#match address VPN_ACL
R2(config-crypto-map)#
```

- ❖ Configuration de l'ACL pour le tunnel VPN :

```
R2(config-crypto-map)#$t ip 192.168.2.0 0.0.0.255 192.168.1.0 0.0.0.255
R2(config)#
```

- ❖ Association à l'interface sortante :

```
R2(config)#int f0/0
R2(config-if)#crypto map VPN_MAP
R2(config-if)#
```

- Vérification des configurations :

❖ Vérification sur le routeur 1 :

```
R1#show running-config interface FastEthernet0/0
Building configuration...

Current configuration : 116 bytes
!
interface FastEthernet0/0
 ip address 192.168.1.1 255.255.255.0
 duplex auto
 speed auto
 crypto map VPN_MAP
end

R1#show running-config | section crypto isakmp
crypto isakmp policy 10
 encr aes
 authentication pre-share
 group 2
crypto isakmp key passer address 192.168.2.1
R1#show running-config | section crypto ipsec
crypto ipsec transform-set TRANSFORM_SET esp-aes esp-sha-hmac
R1#show running-config | section crypto map
crypto map VPN_MAP 10 ipsec-isakmp
 ! Incomplete
 set peer 192.168.2.1
 set transform-set TRANSFORM_SET
 match address VPN_ACL
 crypto map VPN_MAP
R1#
```

❖ Vérification sur le routeur 2 :

```
R2#show running-config interface FastEthernet0/0
Building configuration...

Current configuration : 116 bytes
!
interface FastEthernet0/0
 ip address 192.168.2.1 255.255.255.0
 duplex auto
 speed auto
 crypto map VPN_MAP
end

R2#show running-config | section crypto isakmp
crypto isakmp policy 10
 encr aes
 authentication pre-share
 group 2
crypto isakmp key passer address 192.168.1.1
R2#show running-config | section crypto ipsec
crypto ipsec transform-set TRANSFORM_SET esp-aes esp-sha-hmac
R2#show running-config | section crypto map
crypto map VPN_MAP 10 ipsec-isakmp
 ! Incomplete
 set peer 192.168.1.1
 set transform-set TRANSFORM_SET
 match address VPN_ACL
 crypto map VPN_MAP
R2#
```

III- VPN SITE-TO-SITE AVEC LINUX UBUNTU



Décrivez la configuration d'un VPN IPsec Site-to-Site entre deux serveurs Ubuntu en utilisant Strongswan :

- Configuration côté Serveur A :

1. Configuration d'ISAKMP/IKE :

On édite le fichier de configuration d'ISAKMP (/etc/ipsec.conf ou un fichier spécifique comme /etc/ipsec.d/conn.conf) sur le Serveur A comme ici :

conn site-to-site

```
left=2.2.2.2          # Adresse IP publique du Serveur B
leftsubnet=192.168.2.0/24  # Réseau local du Serveur B
leftfirewall=yes
right=1.1.1.1         # Adresse IP publique du Serveur A
rightsubnet=192.168.1.0/24  # Réseau local du Serveur A
keyexchange=ikev2
authby=secret
auto=start
```

2. Configuration de la clé pré-partagée :

On édite le fichier /etc/ipsec.secrets sur le Serveur B pour spécifier la même clé pré-partagée que sur le Serveur A :

1.1.1.1 2.2.2.2 : PSK "clé_pré-partagée"

Exemple : 1.1.1.1 2.2.2.2 : PSK passer

- Configuration côté Serveur B :

1) Configuration d'ISAKMP/IKE :

On édite le fichier de configuration d'ISAKMP (/etc/ipsec.conf ou un fichier spécifique comme /etc/ipsec.d/conn.conf) sur le Serveur A comme ici :

conn site-to-site

```
left=2.2.2.2          # Adresse IP publique du Serveur B
```

```
leftsubnet=192.168.2.0/24    # Réseau local du Serveur B
leftfirewall=yes
right=1.1.1.1                # Adresse IP publique du Serveur A
rightsubnet=192.168.1.0/24   # Réseau local du Serveur A
keyexchange=ikev2
authby=secret
auto=start
```

2) Configuration de la clé pré-partagée :

On édite le fichier /etc/ipsec.secrets sur le Serveur B pour spécifier la même clé pré-partagée que sur le Serveur A :

```
2.2.2.2 1.1.1.1 : PSK "clé_pré-partagée"
```

Exemple : 2.2.2.2 1.1.1.1 : PSK passer

- Redémarrage des services et vérification :

Après avoir configuré Strongswan sur les deux serveurs, redémarrez le service pour appliquer les configurations :

```
sudo systemctl restart strongswan
```

- Vérification :

Vérifiez l'état de la connexion en utilisant la commande suivante sur chaque serveur :

```
sudo ipsec statusall
```

PARTIE 3 : PKI

I- DIFFÉRENCE ENTRE LES FORMATS DE CERTIFICATS **.P7B, .PFX, .P12, .PEM, .DER, .CRT OU .CER**

Les formats de certificats numériques varient en fonction de leur contenu et de leur utilisation. Le format **** .p7b **** (PKCS#7) contient des certificats et des chaînes de certificats, mais pas de clés privées. Il est utilisé principalement pour l'échange de certificats publics et de chaînes de certificats, ainsi que pour la signature et le chiffrement de données. Ce format est généralement encodé en ASCII (Base64).

Les formats **** .pfx **** (PKCS#12) et **** .p12 **** (PKCS#12) sont similaires et contiennent à la fois des certificats publics et des clés privées. Ils sont utilisés pour importer et exporter des certificats et des clés privées, étant couramment utilisés sous Windows pour des certificats SSL/TLS. Les fichiers .pfx et .p12 sont encodés en binaire, bien que .pfx soit plus courant sur les systèmes Windows et .p12 sur les systèmes Unix/Linux.

Le format **** .pem **** (Privacy Enhanced Mail) est très flexible et peut contenir des certificats, des chaînes de certificats, des clés privées ou des CRL (Certificate Revocation Lists). Il est largement utilisé dans les environnements Unix/Linux et est encodé en ASCII (Base64), avec des en-têtes et des pieds de page comme "-----BEGIN CERTIFICATE-----" et "-----END CERTIFICATE-----".

Le format **** .der **** (Distinguished Encoding Rules) est utilisé lorsqu'un format binaire est requis. Il contient des certificats ou des clés et est couramment utilisé pour importer des certificats dans des systèmes où le format PEM n'est pas supporté.

Enfin, les formats **** .crt **** et **** .cer **** contiennent des certificats X.509. Ils sont utilisés pour les certificats SSL/TLS et les deux extensions sont souvent interchangeables. Toutefois, .crt est plus courant sur les systèmes Unix/Linux et .cer sur les systèmes Windows. Ces formats peuvent être encodés en PEM (ASCII) ou DER (binaire), selon l'application.

II- PKI AVEC OPENSSSL SUR UBUNTU :

- ❖ Créons les répertoires private, cacerts, demoCA, et newcerts sous le répertoire /etc/pki/falilou :

```
root@ubuntu:/home/falilou# mkdir -p /etc/pki/falilou/{private,cacerts,demoCA,newcerts}
root@ubuntu:/home/falilou#
```

- ❖ Créons un fichier index.txt vide puis créons et initialisons le fichier serial:

```
root@ubuntu:/etc/pki/falilou# touch /etc/pki/rtn/demoCA/index.txt
root@ubuntu:/etc/pki/falilou# echo « 01 » > /etc/pki/rtn/demoCA/serial
root@ubuntu:/etc/pki/falilou#
```

- ❖ Générons la clé et le certificat auto-signé pour le CA (racine) avec la commande : **openssl req -x509 -days 3650 -newkey rsa:4096 -keyout private/licencecaKey.pem -out cacerts/licencecaCert.pem -nodes**

```
root@ubuntu:/etc/pki/falilou# openssl req -x509 -days 3650 -newkey rsa:4096 -keyout private/licencecaKey
.pem -out cacerts/licencecaCert.pem -nodes
Generating a RSA private key
.....+++++
.....+++++
writing new private key to 'private/licencecaKey.pem'
-----
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:SN
State or Province Name (full name) [Some-State]:Senegal
Locality Name (eg, city) []:Dakar
Organization Name (eg, company) [Internet Widgits Pty Ltd]:licence
Organizational Unit Name (eg, section) []:licence
Common Name (e.g. server FQDN or YOUR name) []:licence.sn
Email Address []:admin@licence.sn
root@ubuntu:/etc/pki/falilou#
```

- ❖ Générons une requête de signature de certificat pour le serveur Apache/Nginx avec la commande : **openssl req -nodes -newkey rsa:4096 -keyout private/serverKey.key -subj "/C=SN/ST=Senegal/L=dakar/O=licence/CN=licence.sn" -out private/serverReq.csr**

```
root@ubuntu:/etc/pki/falilou# openssl req -nodes -newkey rsa:4096 -keyout private/serverKey.key -subj "/"
C=SN/ST=Senegal/L=dakar/O=licence/CN=licence.sn" -out private/serverReq.csr
Generating a RSA private key
.....+++++
.....+++++
writing new private key to 'private/serverKey.key'
-----
root@ubuntu:/etc/pki/falilou#
```

- ❖ Signature de la requête par le CA avec la commande : **openssl ca -in private/serverReq.csr -days 365 -cert cacerts/licencecaCert.pem -keyfile private/licencecaKey.pem -outdir newcerts/ -out newcerts/serverCert.crt**

```
root@ubuntu:/etc/pki/falilou# openssl ca -in private/serverReq.csr -days 365 -cert cacerts/licencecaCert.pem -keyfile private/licencecaKey.pem -outdir newcerts/ -out newcerts/serverCert.crt
Using configuration from /usr/lib/ssl/openssl.cnf
Check that the request matches the signature
Signature ok
Certificate Details:
  Serial Number: 4096 (0x1000)
  Validity
    Not Before: Jun 25 16:56:19 2024 GMT
    Not After : Jun 25 16:56:19 2025 GMT
  Subject:
    countryName             = SN
    stateOrProvinceName     = Senegal
    organizationName        = licence
    commonName              = licence.sn
  X509v3 extensions:
    X509v3 Basic Constraints:
      CA:FALSE
    Netscape Comment:
      OpenSSL Generated Certificate
    X509v3 Subject Key Identifier:
      04:E8:70:A4:21:97:2D:0E:4A:4D:82:08:5C:42:22:3B:BC:D0:39:6B
    X509v3 Authority Key Identifier:
      keyid:03:9B:76:48:1D:2E:D5:BB:AA:9B:47:68:37:7B:B9:59:59:32:C4:16

Certificate is to be certified until Jun 25 16:56:19 2025 GMT (365 days)
Sign the certificate? [y/n]:y

1 out of 1 certificate requests certified, commit? [y/n]y
Write out database with 1 new entries
Data Base Updated
root@ubuntu:/etc/pki/falilou#
```

- ❖ Générons une requête de signature de certificat pour le client falilou avec la commande : **openssl req -newkey rsa:4096 -subj "/C=SN/ST=Senegal/O=licence/CN=falilou" -keyout private/falilouKey.key -out private/falilouReq.csr**

```
root@ubuntu:/etc/pki/falilou# openssl req -newkey rsa:4096 -subj "/C=SN/ST=Senegal/O=licence/CN=falilou" -keyout private/falilouKey.key -out private/falilouReq.csr
Generating a RSA private key
.....+++++
.....+++++
writing new private key to 'private/falilouKey.key'
Enter PEM pass phrase:
Verifying - Enter PEM pass phrase:
-----
root@ubuntu:/etc/pki/falilou#
```

- ❖ Signature de la requête par le CA avec la commande : **openssl ca -in private/falilouReq.csr -days 365 -out newcerts/falilouCert.crt -notext -cert cacerts/licencecaCert.pem -keyfile private/licencecaKey.pem -outdir newcerts/**


```

root@ubuntu:/etc/pki/falilou# openssl ca -in private/falilouReq.csr -days 365 -out newcerts/falilouCert.crt -notext -cert cacerts/licencecaCert.pem -keyfile private/licencecaKey.pem -outdir newcerts/
Using configuration from /usr/lib/ssl/openssl.cnf
Check that the request matches the signature
Signature ok
Certificate Details:
    Serial Number: 4097 (0x1001)
    Validity
        Not Before: Jun 25 17:02:51 2024 GMT
        Not After : Jun 25 17:02:51 2025 GMT
    Subject:
        countryName           = SN
        stateOrProvinceName   = Senegal
        organizationName      = licence
        commonName            = falilou
    X509v3 extensions:
        X509v3 Basic Constraints:
            CA:FALSE
        Netscape Comment:
            OpenSSL Generated Certificate
        X509v3 Subject Key Identifier:
            CD:CD:FF:3A:D3:CE:28:F7:6A:D1:5D:63:33:B3:80:3E:35:F9:91:96
        X509v3 Authority Key Identifier:
            keyid:03:9B:76:48:1D:2E:D5:BB:AA:9B:47:68:37:7B:B9:59:59:32:C4:16

Certificate is to be certified until Jun 25 17:02:51 2025 GMT (365 days)
Sign the certificate? [y/n]:y

1 out of 1 certificate requests certified, commit? [y/n]:y
Write out database with 1 new entries
Data Base Updated
root@ubuntu:/etc/pki/falilou#

```

❖ Conversion en format p12 pour client windows :

Enregistrons la clé et le certificat du client du fichier au format PKCS#12 qui est un format exécutable sous windows ou linux pour mettre en place facilement le certificat avec la commande : **openssl pkcs12 -export -inkey private/falilouKey.key -in newcerts/falilouCert.crt -certfile cacerts/licencecaCert.pem -out pkcs12/falilou.p12 :**

```

root@ubuntu:/etc/pki/falilou# mkdir /etc/pki/falilou/pkcs12
root@ubuntu:/etc/pki/falilou# openssl pkcs12 -export -inkey private/falilouKey.key -in newcerts/falilouCert.crt -certfile cacerts/licencecaCert.pem -out pkcs12/falilou.p12
Enter pass phrase for private/falilouKey.key:
Enter Export Password:
Verifying - Enter Export Password:
root@ubuntu:/etc/pki/falilou#

```

❖ Importation du certificat sur la machine windows:

The screenshot shows a WinSCP window with two panes. The left pane displays the local file system (C:\Users\sarri\Documents) with various folders and files. The right pane displays the remote file system (/etc/pki/falilou/pkcs12/) containing the falilou.p12 file. The file details for falilou.p12 are shown in the table below.

Nom	Taille	Type	Date de modification	Droits	Propriétés
..		Répertoire parent	25/06/2024 17:32:37		
bank		Dossier de fichiers	28/01/2024 17:58:02		
CALCULATRICE		Dossier de fichiers	23/01/2024 14:09:39		
EC2LT L1		Dossier de fichiers	28/04/2024 20:23:37		
EC2LT L2		Dossier de fichiers	19/06/2024 15:05:29		
formulaire		Dossier de fichiers	23/01/2024 14:09:39		
linphone		Dossier de fichiers	07/06/2024 23:07:56		
Modèles Office perso...		Dossier de fichiers	26/02/2024 09:26:52		
projetsamed		Dossier de fichiers	06/06/2024 14:22:50		
site de bijouterie		Dossier de fichiers	15/04/2024 20:45:13		
tp		Dossier de fichiers	01/06/2024 15:54:21		
falilou.p12	6 KB	Échange d'informat...	25/06/2024 17:12:33	rwxf-rf-x	root
page de garde 2.docx	233 KB	Document Microso...	19/06/2024 16:08:04		
page de garde.docx	161 KB	Document Microso...	08/01/2024 09:56:11		

❖ Activation du module ssl :

```
root@ubuntu:/etc/pki/falilou/pkcs12# a2enmod ssl
Considering dependency setenvif for ssl:
Module setenvif already enabled
Considering dependency mime for ssl:
Module mime already enabled
Considering dependency socache_shmcb for ssl:
Enabling module socache_shmcb.
Enabling module ssl.
See /usr/share/doc/apache2/README.Debian.gz on how to configure SSL and create self-signed certificates.
To activate the new configuration, you need to run:
    systemctl restart apache2
root@ubuntu:/etc/pki/falilou/pkcs12# systemctl restart apache2
root@ubuntu:/etc/pki/falilou/pkcs12#
```

❖ Créons le fichier de configuration du virtualhost ssl :

```
GNU nano 4.8 licence-ssl.conf
<VirtualHost _default_:443>
    ServerName 192.168.1.79
    DocumentRoot /var/www/html/bank

    SSLEngine on
    SSLCertificateFile /etc/pki/falilou/newcerts/serverCert.crt
    SSLCertificateKeyFile /etc/pki/falilou/private/serverKey.key

</VirtualHost>
```

❖ Activons le site :

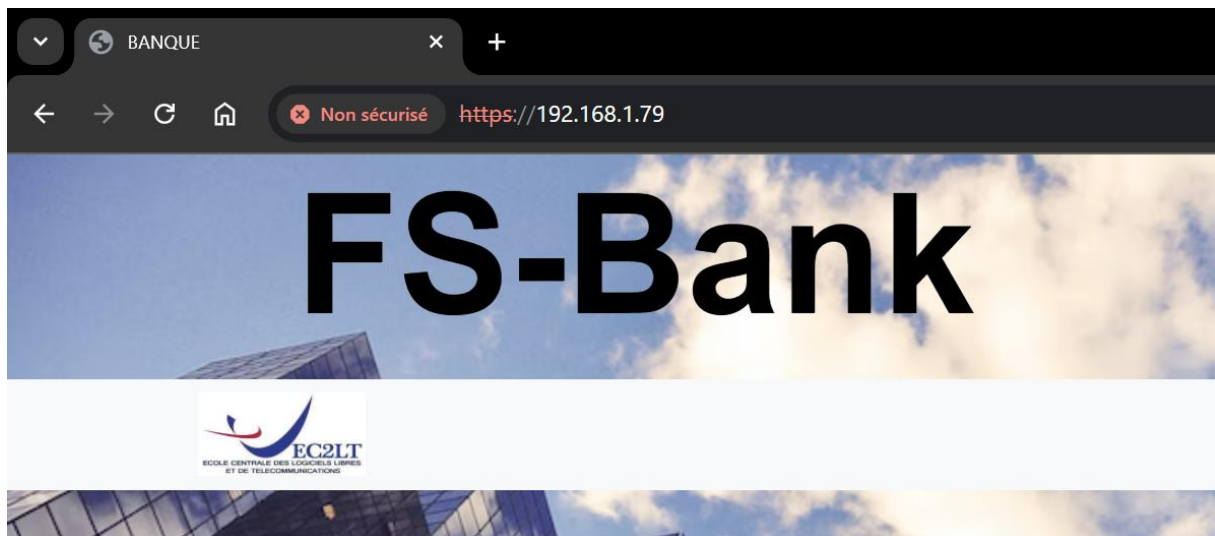
```
root@ubuntu:/etc/apache2/sites-available# a2ensite licence-ssl.conf
Enabling site licence-ssl.
To activate the new configuration, you need to run:
    systemctl reload apache2
root@ubuntu:/etc/apache2/sites-available# sudo systemctl reload apache2
root@ubuntu:/etc/apache2/sites-available#
```

❖ Testons la prise en charge SSL/TLS sur le client sans exiger le certificat client en accédant au site web depuis la machine du client par https :



Le certificat est autosigné du coup le navigateur montre un avertissement de sécurité.

On clique sur continuer vers le site



On accède au site avec https.

- ❖ Pour que le serveur authentifie le client on ajoute les options suivantes dans le fichier de configuration :

```
GNU nano 4.8                               licence-ssl.conf
<VirtualHost _default_:443>
  ServerName 192.168.1.79
  DocumentRoot /var/www/html/bank

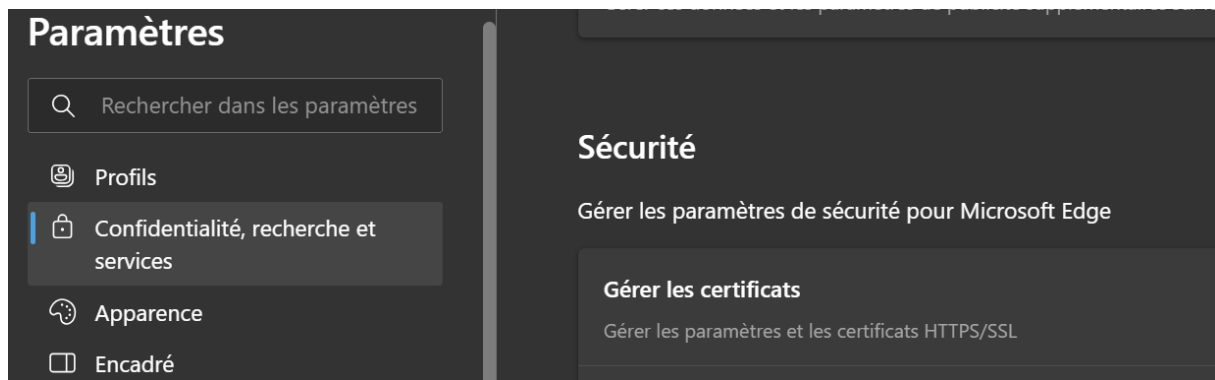
  SSLEngine on
  SSLCertificateFile /etc/pki/falilou/newcerts/serverCert.crt
  SSLCertificateKeyFile /etc/pki/falilou/private/serverKey.key

  SSLVerifyClient require
  SSLVerifyDepth 10
  SSLCACertificateFile /etc/pki/falilou/cacerts/licencecaCert.pem
</VirtualHost>
```

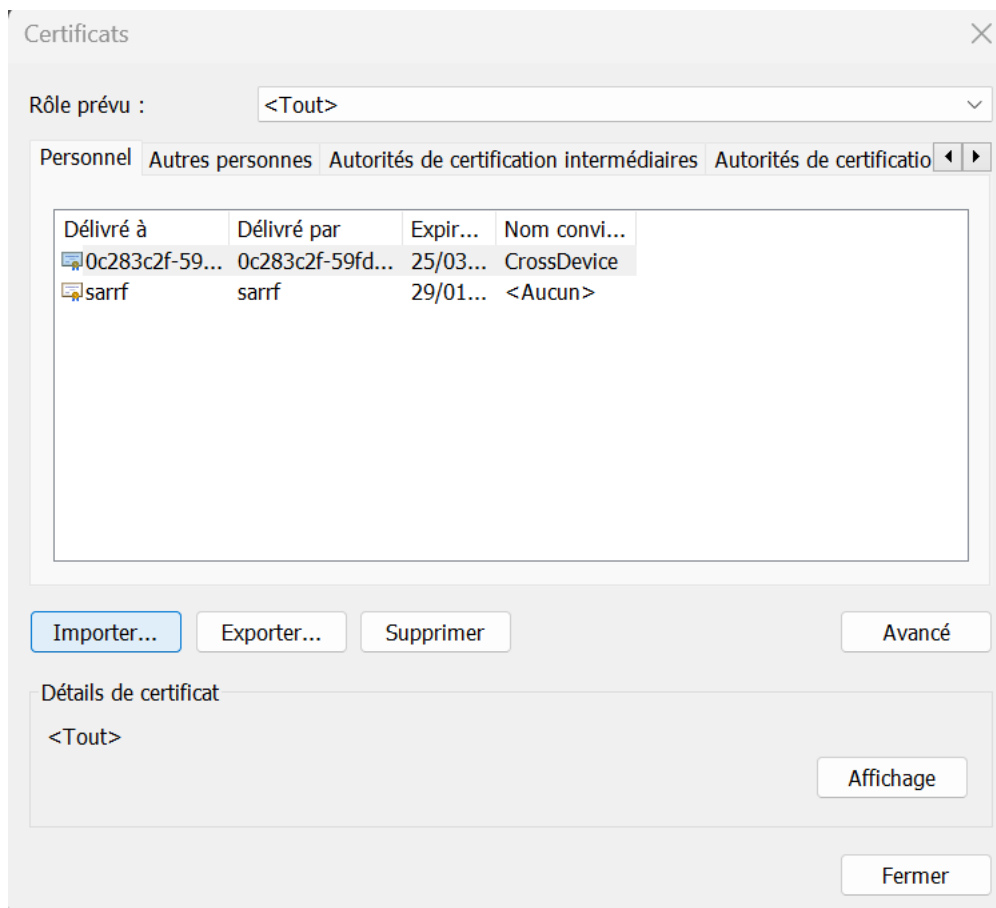
- ❖ Rechargeons apache 2 :

```
root@ubuntu:/etc/apache2/sites-available# sudo systemctl reload apache2
root@ubuntu:/etc/apache2/sites-available#
```

- ❖ Importation/installation du certificat client sur microsoft Edge en se rendant dans Paramètres > Confidentialité, recherche et services > Sécurité > Gérer les certificats :



Puis cliquer sur importer :



Fichier à importer

Spécifiez le fichier à importer.

Nom du fichier :

C:\Users\sarrf\Documents\lalilou.p12

Parcourir...

Remarque : plusieurs certificats peuvent être stockés dans un même fichier aux formats suivants :

Échange d'informations personnelles - PKCS #12 (.PFX,.P12)

Standard de syntaxe de message cryptographique - Certificats PKCS #7 (.P7B)

Magasin de certificats sérialisés Microsoft (.SST)



  Assistant Importation du certificat

Protection de clé privée

Pour maintenir la sécurité, la clé privée a été protégée avec un mot de passe.

Tapez le mot de passe pour la clé privée.

Mot de passe :

☒ Afficher le mot de passe

Options d'importation :

☒ Activer la protection renforcée de clé privée. Une confirmation vous est demandée à chaque utilisation de la clé privée par une application, si vous activez cette option.

☒ Marquer cette clé comme exportable. Cela vous permettra de sauvegarder et de transporter vos clés ultérieurement.

☐ Protéger la clé privée à l'aide de la sécurité par virtualisation (non exportable)

☒ Inclure toutes les propriétés étendues.

Suivant

Annuler

  Assistant Importation du certificat

Magasin de certificats

Les magasins de certificats sont des zones système où les certificats sont conservés.

Windows peut sélectionner automatiquement un magasin de certificats, ou vous pouvez spécifier un emplacement pour le certificat.

- ☐ Sélectionner automatiquement le magasin de certificats en fonction du type de certificat
- ☒ Placer tous les certificats dans le magasin suivant

Magasin de certificats :

Personnel

Parcourir...

Fin de l'Assistant Importation du certificat

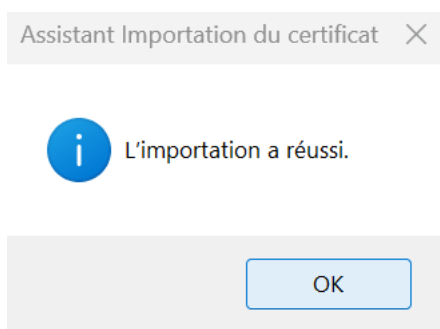
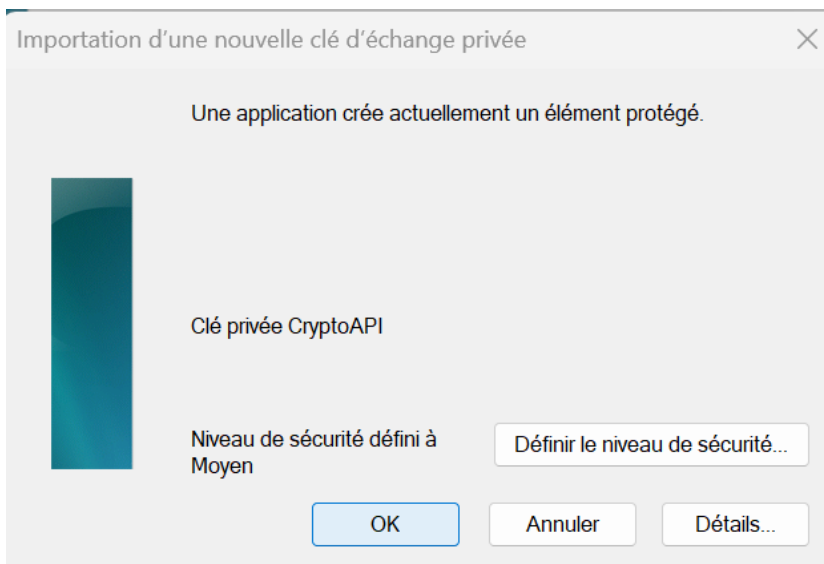
Le certificat sera importé après avoir cliqué sur Terminer.

Vous avez spécifié les paramètres suivants :

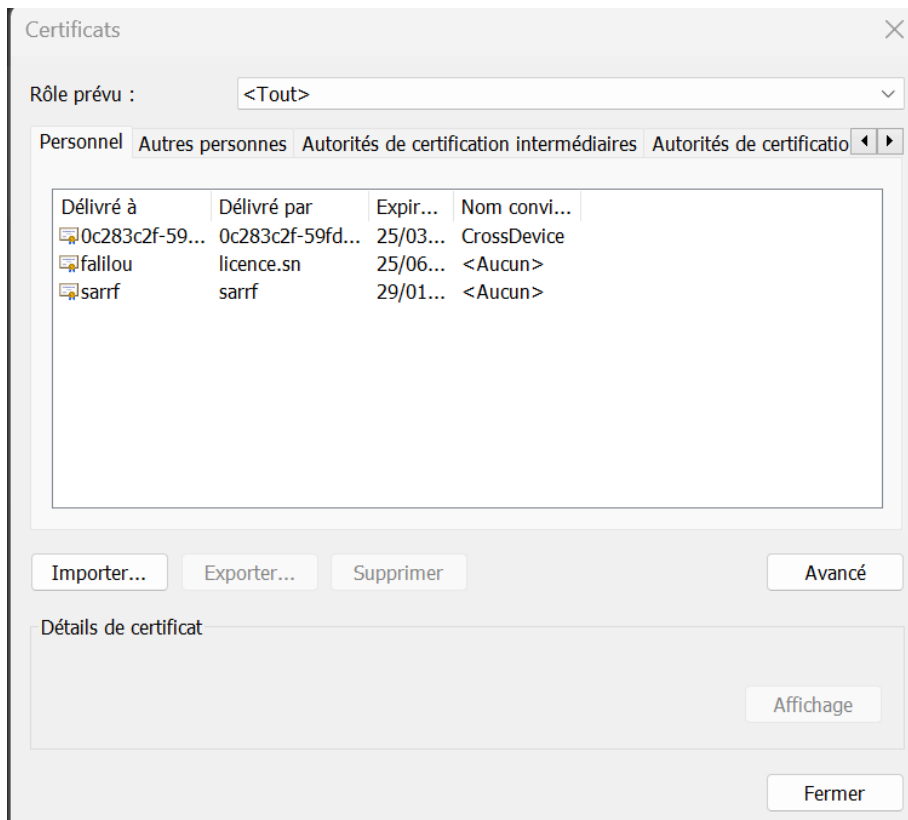
Magasin de certificats sélectionné par l'utilisateur	Personnel
Contenu	PFX
Nom du fichier	C:\Users\sarrf\Documents\falilou.p12

Terminer

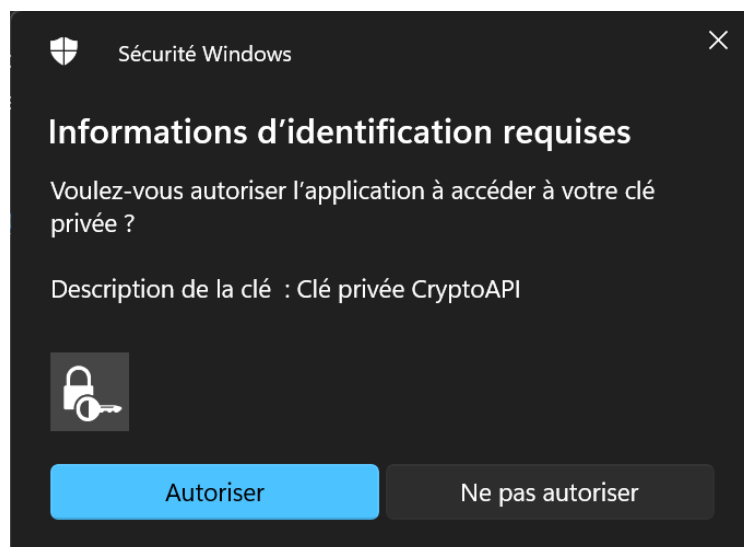
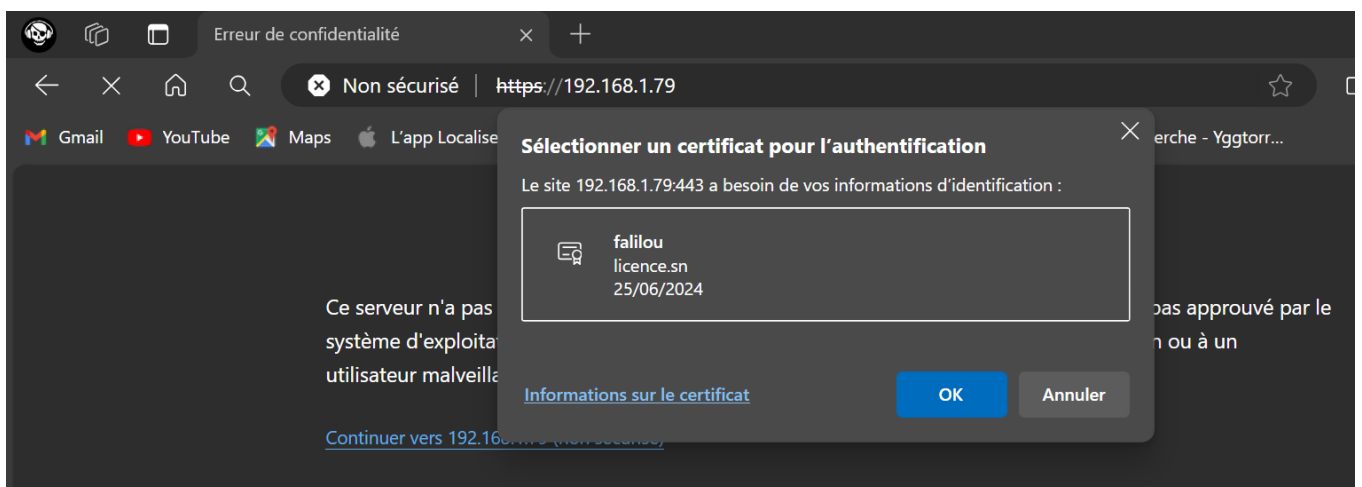
Annuler



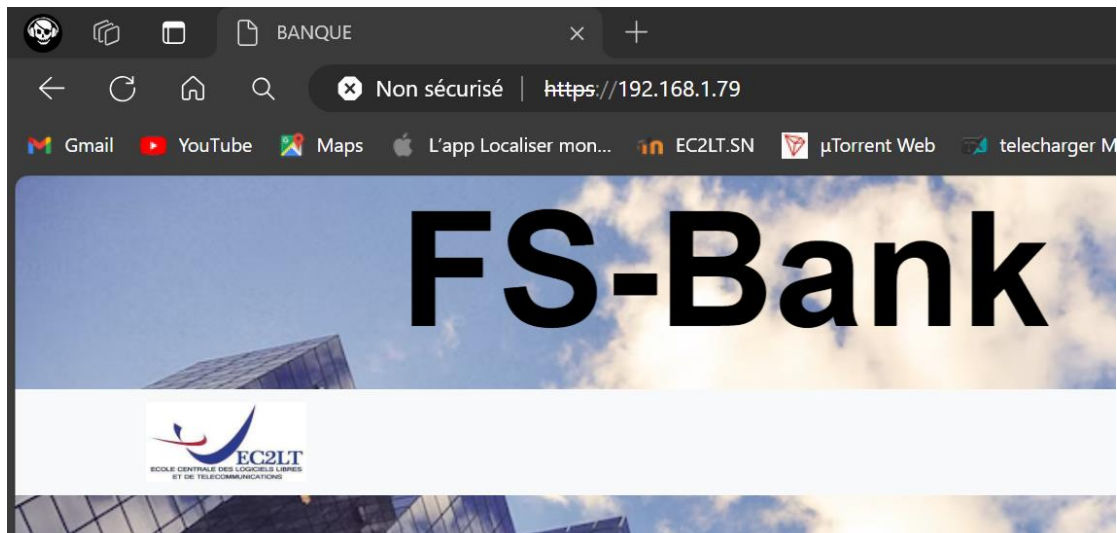
On voit que l'importation est réussi !



❖ Réessayons de nous connecter en fournissant le bon certificat :



Accès au site :



❖ Révocation de certificats et la création d'une liste de révocation de certificats (CRL) :

- Création du dossier contenant les crl et initialisation du numéro de crl :

```
root@ubuntu:/etc/pki/falilou# mkdir crl
root@ubuntu:/etc/pki/falilou# echo 01 > demoCA/crlnumber
root@ubuntu:/etc/pki/falilou#
```

- Générons un crl avec la commande : **openssl ca -cert cacerts/licencecaCert.pem -keyfile private/licencecaKey.pem -gencrl -crldays 15 -out crl/ca.crl**

```
root@ubuntu:/etc/pki/falilou# openssl ca -cert cacerts/licencecaCert.pem -keyfile private/licencecaKey.pem -gencrl -crldays 15 -out crl/ca.crl
Using configuration from /usr/lib/ssl/openssl.cnf
root@ubuntu:/etc/pki/falilou#
```

- Affichons le crl :

```
root@ubuntu:/etc/pki/falilou# openssl crl -in crl/ca.crl -text -noout
Certificate Revocation List (CRL):
  Version 2 (0x1)
  Signature Algorithm: sha256WithRSAEncryption
  Issuer: C = SN, ST = Senegal, L = Dakar, O = licence, OU = licence, CN = licence.sn,
  emailAddress = admin@licence.sn
  Last Update: Jun 25 21:17:55 2024 GMT
  Next Update: Jul 10 21:17:55 2024 GMT
  CRL extensions:
    X509v3 CRL Number:
      1
No Revoked Certificates.
  Signature Algorithm: sha256WithRSAEncryption
  a3:3f:b2:a4:ac:c2:32:cf:3f:4a:7b:8a:81:33:91:51:93:c6:
  4d:7b:c7:be:a8:47:91:22:f4:16:77:85:5a:35:b9:0a:a9:19:
  24:59:7a:6c:63:1b:7e:d1:12:e2:03:a5:71:6c:16:63:f2:03:
  fc:7e:83:07:58:84:28:dd:d6:19:6d:bb:06:50:67:af:33:59:
```

- révoquons le certificat du client pour qu'il n'ait plus accès au site avec la commande : **openssl ca -cert cacerts/licencecaCert.pem -keyfile private/licencecaKey.pem -revoke newcerts/falilouCert.crt**

```
root@ubuntu:/etc/pki/falilou# openssl ca -cert cacerts/licencecaCert.pem -keyfile private/licencecaKey.pem -revoke newcerts/falilouCert.crt
Using configuration from /usr/lib/ssl/openssl.cnf
Revoking Certificate 1001.
Data Base Updated
root@ubuntu:/etc/pki/falilou#
```

- Mise à jour de la liste des certificats révoqués avec la commande : **openssl ca -cert cacerts/licencecaCert.pem -keyfile private/licencecaKey.pem -gencrl -crl days 15 -out crl/ca.crl**

```
root@ubuntu:/etc/pki/falilou# openssl ca -cert cacerts/licencecaCert.pem -keyfile private/licencecaKey.pem -gencrl -crl days 15 -out crl/ca.crl
Using configuration from /usr/lib/ssl/openssl.cnf
root@ubuntu:/etc/pki/falilou#
```

- Vérifions le crl :

```
root@ubuntu:/etc/pki/falilou# openssl crl -in crl/ca.crl -text -noout
Certificate Revocation List (CRL):
  Version 2 (0x1)
  Signature Algorithm: sha256WithRSAEncryption
  Issuer: C = SN, ST = Senegal, L = Dakar, O = licence, OU = licence, CN = licence.sn,
  emailAddress = admin@licence.sn
  Last Update: Jun 25 21:24:59 2024 GMT
  Next Update: Jul 10 21:24:59 2024 GMT
  CRL extensions:
    X509v3 CRL Number:
      2
Revoked Certificates:
  Serial Number: 1001
  Revocation Date: Jun 25 21:22:54 2024 GMT
  Signature Algorithm: sha256WithRSAEncryption
    ca:e7:4d:e1:3f:1b:50:b7:4b:19:fc:d8:20:67:3f:5d:09:a8:
    2f:0b:c6:2e:1f:2a:a6:ee:ea:b2:3c:72:71:24:26:f3:94:c2:
```

Maintenant on voit **Revoked Certificates** et un **Serial Number : 1001**

- Ajoutons les paramètres de révocation dans notre configuration apache2 :

```
GNU nano 4.8                               licence-ssl.conf
<VirtualHost _default_:443>
  ServerName 192.168.1.79
  DocumentRoot /var/www/html/bank

  SSLEngine on
  SSLCertificateFile /etc/pki/falilou/newcerts/serverCert.crt
  SSLCertificateKeyFile /etc/pki/falilou/private/serverKey.key

  SSLCARevocationCheck chain
  SSLCARevocationFile /etc/pki/falilou/crl/ca.crl

  SSLVerifyClient require
  SSLVerifyDepth 10
  SSLCACertificateFile /etc/pki/falilou/cacerts/licencecaCert.pem
</VirtualHost>
```

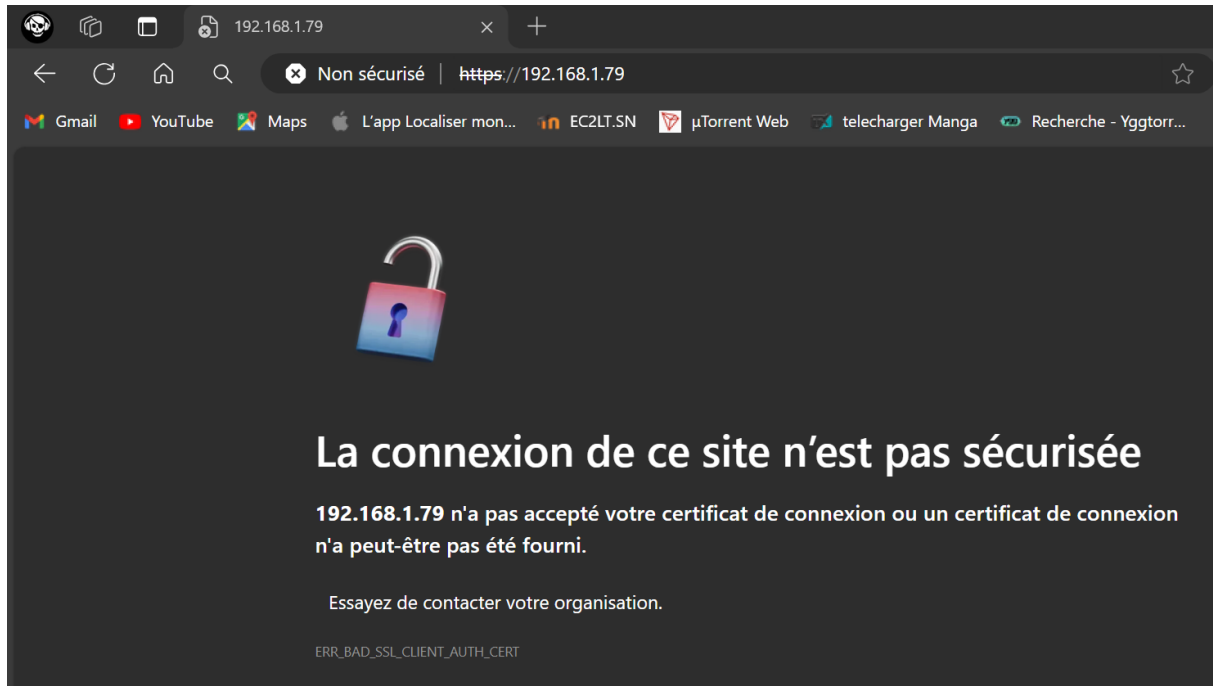

SSLCARevocationCheck => Active la vérification des révocations basée sur les CRL.

SSLCARevocationFile => Fichier contenant la concaténation des CRLs des CA pour l'authentification des clients.

- Rechargeons le service apache2 :

```
root@ubuntu:/etc/apache2/sites-available# sudo systemctl reload apache2
root@ubuntu:/etc/apache2/sites-available#
```

- Essayons à présent de se reconnecter au site :



On voit qu'on ne peut plus accéder au site car le certificat a été révoqué.

CONCLUSION

Ce rapport couvre trois parties principales : la cryptographie, la sécurité des LAN et la mise en place d'une PKI avec OpenSSL sur Ubuntu. La première partie traite des fonctions de hachage, des différences entre cryptographie symétrique et asymétrique, et inclut des exercices pratiques sur l'utilisation des clés RSA et AES-256-CBC. La deuxième partie détaille la configuration des VPN d'accès à distance et Site-to-Site sur des routeurs Cisco, ainsi que la configuration d'un VPN IPsec Site-to-Site entre des serveurs Ubuntu. La dernière partie explique les différents formats de certificats et fournit un guide pour créer une PKI, incluant la génération et la signature de certificats et le processus de révocation.